

UNIVERSIDADE FEDERAL DE ALFENAS (UNIFAL-MG)
Instituto de Ciências Exatas (ICEEx)
Disciplina de Redes de Computadores

TRABALHO FINAL
Implementação do Protocolo Go-Back-N (GBN) via
UDP

Autores:

Flávia Marcella Gonçalves Moreira
Renan Catini Amaral

Alfenas, MG
1 de julho de 2026

Resumo

Este relatório descreve as decisões de projeto, a implementação e a análise de desempenho do protocolo de transferência confiável Go-Back-N (GBN) operando sobre a camada UDP. O sistema foi desenvolvido em Java e testado sob diferentes condições de rede, variando-se o tamanho da janela de transmissão (N) e a probabilidade de perda de pacotes. Os resultados demonstram a eficácia do temporizador, dos ACKs cumulativos e do buffer circular para garantir a entrega íntegra de arquivos binários.

Conteúdo

1	Introdução	3
2	Decisões de Projeto e Implementação	3
2.1	Estrutura do Cabeçalho	3
2.2	Gerenciamento da Janela e Buffers	3
2.3	Temporização e Concorrência	3
3	Metodologia de Testes	4
4	Resultados e Discussões	4
4.1	Impacto do Pipelining e o Gargalo do Stop-and-Wait	4
4.2	Comportamento sob Taxas de Perdas	5
4.3	Análise do Efeito "Localhost" e ACKs Cumulativos	5
4.4	Validação Estatística da Simulação de Perda	5
5	Conclusão	6

1 Introdução

O User Datagram Protocol (UDP) é um protocolo de transporte leve e rápido, porém não confiável, pois não oferece garantias de entrega, ordenação ou controle de congestionamento. Para a transferência íntegra de arquivos, torna-se necessário implementar mecanismos de confiabilidade na camada de aplicação.

Este trabalho tem como objetivo implementar o protocolo Go-Back-N (GBN), que utiliza uma abordagem de requisição de repetição automática (ARQ) com *pipeline*.

2 Decisões de Projeto e Implementação

A arquitetura do projeto foi dividida em duas entidades independentes: o Emissor (*Sender*) e o Receptor (*Receiver*).

2.1 Estrutura do Cabeçalho

Para simular a segmentação de dados, estipulou-se um *payload* máximo de 1024 bytes por pacote, precedido por um cabeçalho customizado de 11 bytes utilizando a classe `ByteBuffer` do Java. A estrutura adotada foi:

- **Tipo (1 byte):** 0 (Dados), 1 (ACK), 2 (Handshake), 3 (Fim).
- **Número de Sequência (4 bytes):** Inteiro que identifica o pacote.
- **Número de ACK (4 bytes):** Inteiro para confirmação cumulativa.
- **Tamanho dos Dados (2 bytes):** Define o tamanho útil do *payload*.

2.2 Gerenciamento da Janela e Buffers

No Emissor, implementou-se um **buffer circular** na forma de um *array* unidimensional de tamanho N . O acesso circular (através do operador módulo $\% N$) garantiu a reusabilidade da memória sem a necessidade de realocação dinâmica contínua durante transferências de grandes arquivos.

2.3 Temporização e Concorrência

A recepção de confirmações (ACKs) no Emissor foi tratada através de uma *Thread* dedicada, permitindo o comportamento *full-duplex* do protocolo: enquanto a *thread* principal lê o arquivo em disco e envia novos segmentos, uma *thread* secundária permanece bloqueada em `socket.receive()`, processando ACKs de forma assíncrona e avançando a janela de transmissão sem interromper o fluxo de envio.

O controle do temporizador de *timeout* foi implementado com as classes `Timer` e `TimerTask` do Java, encapsuladas em métodos `synchronized` (`iniciarTimer` e `pararTimer`) para evitar condições de corrida entre a *thread* de envio (que inicia o cronômetro ao transmitir o primeiro pacote de uma janela vazia) e a *thread* de recepção de ACKs (que reinicia ou cancela o cronômetro conforme a base da janela avança).

Adicionalmente, identificou-se que o *array* do buffer circular (`janela[]`) é acessado concorrentemente por duas *threads* distintas: a *thread* principal, que grava novos pacotes enviados (`janela[nextseqnum % N] = dataPacket`), e a *thread* do `Timer`, que lê

esse mesmo *array* para retransmitir pacotes em caso de *timeout*. Como o modificador `volatile` aplicado às variáveis `base` e `nextseqnum` garante visibilidade apenas para valores primitivos individuais – e não para o conteúdo de um *array* – foi introduzido um objeto de *lock* dedicado (`janelaLock`), utilizado tanto na escrita quanto na leitura do buffer circular. Essa medida assegura a corretude do Modelo de Memória do Java (JMM) nas transições entre *threads*, prevenindo leituras inconsistentes durante a retransmissão.

3 Metodologia de Testes

Os testes foram conduzidos utilizando a interface de *loopback* (localhost) sob cenários controlados pelo próprio código para forçar a simulação de perdas de pacotes.

Foi selecionado um arquivo de teste de tamanho aproximado de 3 MB. O Receptor foi inicializado com o comando `java Receiver 5000`, escutando na porta 5000, e o Emissor com o comando `java Sender <arquivo> 127.0.0.1:<destino> <N> <prob_perda>`, variando-se os parâmetros N (tamanho da janela) e `prob_perda` (probabilidade de perda simulada) entre execuções.

A cada bateria de testes, foram coletadas as seguintes métricas, exibidas ao final da execução de cada módulo: (i) tempo total de transferência, cronometrado do primeiro envio de *handshake* até o recebimento do sinal de FIM; (ii) **throughput** médio, calculado em tempo real durante a transferência e reportado ao final como a razão entre o total de bytes efetivamente transmitidos (incluindo retransmissões) e o tempo decorrido; (iii) número de eventos de *timeout* e de pacotes individualmente retransmitidos pelo Emissor; e (iv) no Receptor, o número de pacotes recebidos corretamente em ordem, o número de pacotes descartados pela simulação de perda, e o número de pacotes fora de ordem ou duplicados – estes últimos contabilizados separadamente para não distorcer a taxa de perda efetiva, conforme detalhado na Seção 4.4.

4 Resultados e Discussões

A fim de validar a implementação e observar a teoria do protocolo na prática, o sistema foi submetido a baterias de testes transferindo um arquivo `teste.pdf` de quase 3MB via interface de **loopback**. Os cenários variaram o tamanho da janela de transmissão (N) e a probabilidade de descarte de pacotes (simulada).

4.1 Impacto do Pipelining e o Gargalo do Stop-and-Wait

A variação do tamanho da janela (N) demonstrou categoricamente a eficácia do uso de **pipelining** para maximizar a utilização do canal.

- Com $N = 1$ - com perda **0.0** (equivalente ao protocolo **Stop-and-Wait**), o tempo de transferência totalizou exatos **32.163s**.
- Ao ampliar a janela para $N = 5$ (com perda **0.0**), o tempo caiu drasticamente para **1.774s**.
- Com $N = 50$ - com perda **0.0**, o tempo reduziu-se a expressivos **0.324s**.

Fica evidente que transmitir múltiplos pacotes em sequência sem aguardar bloqueios individuais de confirmação aumenta substancialmente o **throughput** da aplicação. A janela $N = 50$ foi aproximadamente 100 vezes mais rápida que o modelo **Stop-and-Wait**.

4.2 Comportamento sob Taxas de Perdas

Ao introduzir taxas de descarte de pacotes, foi possível avaliar o impacto dos **timeouts** e das retransmissões inerentes ao Go-Back-N.

- Com $N = 5$ e perda de 10% (0.1), o tempo subiu de 1.774s para **18.117s**. O atraso é justificado pela necessidade de aguardar o temporizador (0.1 segundo) a cada perda confirmada, retransmitindo a janela estreita.
- Com $N = 50$, o comportamento surpreendeu: sob 10% de perda o tempo foi de **15.912s**, e sob uma taxa severa de 30% de perda, o tempo foi de **64.744s**.

4.3 Análise do Efeito "Localhost" e ACKs Cumulativos

A resiliência da janela $N = 50$ sob perdas de até 30% deve-se a dois fatores observados durante a execução:

1. **ACK Cumulativo em Ação:** Como o GBN utiliza confirmações cumulativas, a perda de dezenas de ACKs sequenciais na janela de 50 foi facilmente mascarada pelo recebimento de um único ACK posterior, que confirmou todo o lote pendente e evitou o estouro do temporizador.
2. **Penalidade da Retransmissão Massiva:** Embora o ambiente de *loopback* possua atraso de propagação nulo, a taxa de 30% de perda evidenciou a maior fraqueza teórica do Go-Back-N. Cada pacote descartado forçou o sistema a retransmitir uma janela inteira de 50 pacotes. Essa retransmissão massiva e redundante gerou um *overhead* extremo, fazendo o tempo saltar para quase 65 segundos e provando na prática que janelas muito amplas são altamente ineficientes em redes ruidosas.

4.4 Validação Estatística da Simulação de Perda

Conforme especificado no enunciado do trabalho, a simulação de perda no Receptor deve atuar exclusivamente sobre pacotes de dados recebidos corretamente **em ordem**; pacotes fora de ordem ou duplicados – já descartados pela própria lógica do GBN, independentemente da simulação – não devem ser contabilizados na taxa de perda efetiva. Para atender a essa exigência, o Receptor separa dois contadores distintos: `totalPacotesRecebidos`, incrementado apenas quando `numSeq == expectedseqnum` (isto é, pacotes candidatos à simulação), e `pacotesForaDeOrdem`, que registra descartes naturais do protocolo sem influenciar a estatística de perda. A taxa de perda efetiva é então calculada como:

$$\text{Taxa de perda efetiva} = \frac{\text{pacotesDescartados}}{\text{totalPacotesRecebidos}} \quad (1)$$

Essa separação foi validada empiricamente em uma execução de teste com N elevado e probabilidade de perda configurada em 10% ($p = 0,1$), cujos resultados de ambos os módulos são reproduzidos na Tabela 1.

Tabela 1: Estatísticas de uma execução com probabilidade de perda configurada em 10%

Métrica	Valor
Pacotes originais enviados pelo Emissor	2871
Pacotes individuais retransmitidos pelo Emissor	15643
Total de pacotes enviados fisicamente	18514
Pacotes recebidos em ordem (candidatos à simulação)	3184
Pacotes descartados pela simulação de perda	313
Pacotes fora de ordem/duplicados (Receptor)	15330
Taxa de perda efetiva calculada	9,83%
Probabilidade de perda configurada	10,00%

Dois pontos confirmam a corretude da implementação. Primeiro, a taxa de perda efetiva (9,83%) converge para a probabilidade configurada (10%), conforme esperado pela Lei dos Grandes Números à medida que o número de pacotes transferidos cresce. Segundo, verificam-se as seguintes identidades de conservação entre os dois módulos:

$$\underbrace{2871}_{\text{originais}} + \underbrace{313}_{\text{descartados pela simulação}} = \underbrace{3184}_{\text{candidatos em ordem}} \quad (2)$$

$$\underbrace{2871 + 15643}_{\text{total enviado pelo Emissor}} = \underbrace{3184 + 15330}_{\text{total processado pelo Receptor}} = 18514 \quad (3)$$

Como o ambiente de teste é um enlace de *loopback*, sem perda real de rede, todo pacote transmitido fisicamente pelo Emissor deve necessariamente ser contabilizado em alguma das categorias do Receptor – o que se confirma pela igualdade acima. Essa verificação cruzada demonstra que a separação entre "perda simulada" e "descarte por desordem/duplicidade" está corretamente implementada, sem sobreposição ou perda de contabilidade entre as duas categorias.

Em suma, os resultados ratificam a robustez do protocolo desenvolvido, atestando a correção da máquina de estados, o sucesso na ordenação dos dados no disco e a efetividade dos temporizadores assíncronos.

5 Conclusão

A implementação do Go-Back-N em Java evidenciou os desafios do desenvolvimento de protocolos na camada de aplicação. Gerenciar a concorrência entre threads de leitura e escrita e coordenar o acesso ao disco para evitar corrompimento de arquivos exigiu um projeto minucioso.

O sistema atendeu a todos os requisitos propostos, transferindo arquivos binários de maneira fidedigna. Verificou-se na prática que o protocolo é funcional, embora sofra grande impacto de desempenho em redes ruidosas devido à sua natureza de retransmitir toda a janela a cada **timeout**, reforçando os conceitos estudados em aula.